

Ejercicio Nro: 13

Enunciado

- 1- ¿Qué es Extreme Programming (XP) y cuál es su objetivo principal dentro de las metodologías ágiles?
- 2- ¿Cuáles son los cinco valores principales de XP? Explicá brevemente cada uno.
- 3- ¿Por qué XP considera que las pruebas son un elemento fundamental del desarrollo de software?
- 4- ¿Qué es Test Driven Development (TDD) y cómo se relaciona con XP?
- 5- ¿En qué consiste la práctica de Pair Programming? Mencioná dos ventajas y una posible dificultad.
- 6- ¿Qué son las historias de usuario en XP y por qué se prefieren frente a una especificación extensa de requisitos?
- 7- ¿Qué significa Continuous Integration en XP y qué beneficios aporta al equipo de desarrollo?
- 8- ¿Cómo se aplica el concepto de Weekly Cycle en un proyecto desarrollado con XP?
- 9- En XP se plantea que se fija el tiempo, el costo y la calidad, y se negocia el alcance. ¿Qué significa esta idea? Explicalo con un ejemplo.
- 10- Elegí tres prácticas de XP y explicá cómo podrían aplicarse en un proyecto real de desarrollo de software.

Resolución

1.

Es un enfoque de desarrollo de software que se centra en un proceso adaptativo y orientado a las personas.

Objetivo principal: Sus promesas y objetivos principales son reducir el riesgo del proyecto, mejorar la respuesta ante los cambios en el negocio, mejorar la productividad a lo largo de la vida del software y hacer más divertido el trabajo del equipo construyendo el software.

2.

Los 5 valores fundamentales que guían la metodología son:

- **Comunicación:** Fomenta el intercambio constante de información dentro del equipo y con el cliente.
- **Simplicidad:** Se busca diseñar y programar solo lo que es necesario en el momento, evitando complejidades innecesarias.

- **Retroalimentación (Feedback):** Aprender constantemente de las pruebas, del sistema y del cliente para realizar ajustes inmediatos.
- **Valentía (Coraje):** Capacidad para tomar decisiones firmes, como aceptar cambios, rehacer código si es necesario o enfrentar problemas complejos sin miedo.
- **Respeto:** Valorar a los miembros del equipo, sus aportes y asegurar que todos se sientan respetados en su trabajo.

3.

XP sostiene que las pruebas son un pilar básico porque sirven para dar certidumbre y asegurar la calidad del código. El documento destaca que:

- Son el elemento fundamental del desarrollo y **todos los desarrolladores deben escribirlas** mientras escriben el código que finalmente irá a producción.
- Se integran en un proceso de integración y construcción continua, lo que genera una plataforma muy estable para el desarrollo futuro.
- Ofrecen confianza y ritmo de trabajo; si un test es difícil de escribir, suele ser una señal de que hay un problema de diseño (código acoplado).

4.

- **Qué es:** Es el desarrollo guiado por pruebas (también llamado *Test-First Programming*), una práctica donde se escriben las pruebas automáticas **antes** de escribir cualquier línea de código de producción.
- **Relación con XP:** TDD es una de las prácticas esenciales (primarias) de XP. Sigue un ciclo muy marcado de tres pasos: escribir una prueba que falle (**Red**), escribir el código mínimo para que la prueba pase (**Green**), y finalmente limpiar y optimizar el código (**Refactor**). Kent Beck, creador de XP, es además una de las figuras clave en el desarrollo de esta práctica y de herramientas como JUnit.

5.

- **En qué consiste:** Todo el código que va a producción debe ser escrito por dos personas sentadas frente a la misma máquina. Es un diálogo constante entre dos programadores que están simultáneamente analizando, diseñando, probando e intentando programar mejor.
- Ventajas:
 - 1-Se mantienen centrados mutuamente.

2-Se cumplen mejor los estándares del equipo.

- **Posible dificultad:** El documento menciona como consejo/advertencia "No invadir el espacio personal del otro (monitores grandes)", lo que refleja que la cercanía física y la falta de espacio adecuado pueden generar incomodidad o fricciones. También advierte que no se deben juntar dos programadores novatos.

6.

- **Qué son:** Son descripciones breves de funcionalidades escritas en tarjetas pequeñas, que incluyen un nombre, una descripción y una estimación de tiempo.
- **Por qué se prefieren:** Se prefieren porque el término "requisitos" tiene una connotación de "inmutabilidad" y "permanencia" que choca con la filosofía ágil de "abrazar el cambio". Al usar historias en tarjetas pequeñas (y colocarlas a la vista en la pared, no en un programa), es mucho más fácil estimar el coste de cada una, moverlas, priorizarlas y negociar el alcance de forma flexible.

7.

Significa no dejar pasar más de dos horas sin integrar en la base de código común los cambios que los programadores han desarrollado. Cada pareja, tras un par de horas de trabajo, sube sus cambios para que se complete el "build" (construcción) y se ejecuten todos los tests automáticos.

- **Beneficios:**
 - Rompe el problema de "divide, vencerás e integrarás" (evita que la integración se vuelva un caos impredecible al final).
 - Asegura que no haya problemas de regresión (que algo nuevo rompa lo que ya funcionaba).
 - Permite detectar errores de forma inmediata; si el build diario falla o se "notifica con alertas", el equipo se entera al instante para solucionarlo.
 - Mantiene el sistema siempre listo para ser desplegado en producción.

8.

El ciclo semanal se aplica como el motor del progreso a corto plazo mediante los siguientes pasos:

1. **Reunión al comienzo de cada semana:** Se revisa el progreso hasta la fecha (incluyendo si el progreso real se corresponde con lo planificado).
2. **Selección de Historias:** El cliente elige qué historias de usuario (que sumen el equivalente a una semana de trabajo) se van a realizar en esos días.
3. **Fraccionamiento y Estimación:** Los desarrolladores dividen esas historias en tareas técnicas más chicas, se las reparten y las estiman.

4. **Desarrollo y Pruebas:** Se comienza la semana escribiendo las pruebas automáticas de las historias y se progresa implementándolas.
5. **Fin del ciclo:** Al terminar la semana, las nuevas historias tienen que estar completamente listas para ser desplegadas.

9.

En el "triángulo de hierro" tradicional (tiempo, coste, alcance), bajar la calidad para cumplir los plazos es una pésima decisión porque genera "deuda técnica". XP propone que el Tiempo de entrega, el Costo (recursos/equipo) y la Calidad del software no se tocan (son fijos). Por lo tanto, si el tiempo aprieta, lo único que se puede ajustar es el **Alcance** (es decir, qué tantas cosas o funcionalidades se van a entregar en esa fecha).

- **Ejemplo:** Imaginemos que estamos desarrollando una tienda online y la fecha de lanzamiento (Tiempo) es en dos semanas, el presupuesto y los programadores (Costo) están definidos, y el sistema debe funcionar perfectamente sin errores (Calidad). Si vemos que no llegamos a programar todo, no compaginamos el código a las apuradas (lo que rompería la calidad); en su lugar, nos reunimos con el cliente y negociamos: *"En dos semanas lanzamos la web con el carrito y pagos con tarjeta (Alcance cubierto), pero dejamos la sección de cupones de descuento y pagos con criptomonedas para la siguiente entrega (Alcance postergado)"*.

10.

1. Informative Workspace (Espacio de trabajo informativo):
 - *Aplicación real:* En la oficina del equipo de desarrollo, se destina una pizarra física o pared principal (visible para todos) donde se pegan tarjetas (post-its) divididas en columnas como *"Por hacer"*, *"Esta semana"*, *"Hecho"*. Además, se cuelga un monitor que muestre un gráfico de barras con la evolución del proyecto en tiempo real. Así, cualquiera que pase sabe exactamente el estado actual del software.
2. Ten-Minute Build (Construcción en 10 minutos):
 - *Aplicación real:* En un proyecto web, se configura un script automatizado (como en GitHub Actions). Cada vez que un desarrollador termina una tarea, sube su código y el servidor automáticamente compila el proyecto, levanta la base de datos de prueba y ejecuta los cientos de tests en menos de 10 minutos. Si algo se rompe, el sistema avisa enseguida y el equipo lo corrige antes de avanzar.
3. Energized Work (Trabajo energizado):
 - *Aplicación real:* Para evitar que los desarrolladores se quemén y escriban código defectuoso por cansancio, el líder de proyecto prohíbe las horas extras

sistemáticas y fomenta la jornada estándar de 40 horas. En el día a día, se implementa una regla: de 14:00 a 16:00 hs se declara "Tiempo de Programación enfocado", donde todos apagan las notificaciones de Slack, no ven emails y se concentran al 100% solo en tirar código sin interrupciones.

Ejercicio Nro: 13 Desarrollo de Aplicación Bancaria con TDD

Enunciado:

Tu tarea es desarrollar una aplicación informática utilizando la técnica TDD para gestionar una cuenta bancaria. La aplicación debe permitir a los usuarios abrir una cuenta, realizar depósitos, hacer retiros y transferir fondos entre cuentas. A continuación se detallan las etapas de desarrollo utilizando TDD:

Etapas 1: Especificación y prueba inicial

1. Especifica los requisitos básicos del sistema y las funcionalidades clave, como la apertura de cuenta, depósito de fondos, retiro de fondos y transferencia de fondos.
2. Escribe una prueba inicial que verifique si el sistema puede crear una instancia de una cuenta bancaria y obtener su saldo inicial.

Etapas 2: Desarrollo de las funcionalidades básicas

3. Implementa la funcionalidad para abrir una cuenta bancaria, asegurándote de que se cumplan los requisitos especificados. Ejecuta la prueba y verifica que pase correctamente.
4. Implementa la funcionalidad para realizar depósitos en una cuenta bancaria. Ejecuta las pruebas y verifica que pasen correctamente.
5. Implementa la funcionalidad para realizar retiros de una cuenta bancaria. Ejecuta las pruebas y verifica que pasen correctamente.
6. Implementa la funcionalidad para transferir fondos entre cuentas bancarias. Ejecuta las pruebas y verifica que pasen correctamente.

Etapas 3: Pruebas adicionales y mejoras

7. Escribe pruebas adicionales para cubrir casos de prueba específicos, como intentar retirar más dinero del disponible en una cuenta o transferir fondos a una cuenta inexistente.
8. Ejecuta todas las pruebas y verifica que pasen correctamente.
9. Refactoriza tu código si es necesario para mejorar su estructura, legibilidad y eficiencia.
10. Ejecuta todas las pruebas nuevamente para asegurarte de que el código refactorizado no haya introducido errores.

Etapas 4: Cobertura completa de pruebas

11. Asegúrate de que todas las funcionalidades del sistema estén cubiertas por pruebas automatizadas.
12. Examine los casos límite y situaciones excepcionales para garantizar que el sistema se comporte correctamente en todos los escenarios.
13. Ejecuta todas las pruebas y verifica que pasen correctamente.

Recuerda seguir el enfoque TDD, donde agregarás una prueba antes de implementar cada funcionalidad y verificarás que todas las pruebas pasen antes de pasar a la siguiente etapa. Esto te ayudará a desarrollar una aplicación confiable, mantenible y que cumpla con los requisitos establecidos.

Resolución

Etapas 1: Especificación y prueba inicial

1. Especificación de requisitos básicos y functionalities clave

- **Apertura de cuenta:** Creación de una instancia de cuenta asociada a un identificador único y un titular, con la opción de asignar un saldo inicial.
- **Depósito de fondos:** Incremento del saldo disponible mediante el ingreso de un monto válido y positivo.
- **Retiro de fondos:** Reducción del saldo mediante una extracción, validando que no se supere el dinero disponible ni se ingresen montos negativos o iguales a cero.
- **Transferencia de fondos:** Traspaso de dinero desde una cuenta origen hacia una cuenta destino de forma segura y atómica.

2. Prueba inicial (Ciclo TDD - Fase Roja)

Objetivo: Validar la creación correcta de la entidad y la lectura del saldo inicial asignado antes de escribir el código de producción.

JavaScript

```
// archivo: cuentaBancaria.test.js
const assert = require('assert');
const { CuentaBancaria } = require('./cuentaBancaria');

try {
  console.log("Ejecutando Prueba Inicial: Creación de Cuenta...");
  const cuenta = new CuentaBancaria("12345", "Carlos Gómez", 500.0);

  assert.strictEqual(cuenta.numeroCuenta, "12345");
  assert.strictEqual(cuenta.obtenerSaldo(), 500.0);
  console.log("✓ Prueba superada.");
} catch (error) {
  console.error("✗ Resultado esperado en TDD (Fase Roja): La prueba falla porque la clase aún no existe.");
  console.error(error.message);
}
```

Etapla 2: Desarrollo de las funcionalidades básicas

3. Implementación para abrir una cuenta bancaria (Fase Verde)

Escribimos el código de producción mínimo indispensable en un archivo centralizado para que la prueba inicial pase a verde.

JavaScript

```
// archivo: cuentaBancaria.js
class CuentaBancaria {
  constructor(numeroCuenta, titular, saldoInicial = 0) {
    this.numeroCuenta = numeroCuenta;
    this.titular = titular;
    this.saldo = saldoInicial >= 0 ? saldoInicial : 0;
  }

  obtenerSaldo() {
    return this.saldo;
  }
}

module.exports = { CuentaBancaria };
```

Resultado: Al ejecutar la prueba inicial con este archivo creado, el resultado pasa a **Verde** exitosamente.

4. Implementación para realizar depósitos

- **Código de la prueba (Fase Roja):**

JavaScript

```
const cuenta = new CuentaBancaria("12345", "Carlos Gómez", 500.0);
cuenta.depositar(200.0);
assert.strictEqual(cuenta.obtenerSaldo(), 700.0);
// Error: cuenta.depositar is not a function
```

- **Código de producción (Fase Verde):** Añadimos el método a nuestra clase.

JavaScript

```
// Se agrega dentro de class CuentaBancaria
depositar(monto) {
  if (monto <= 0) {
    throw new Error("El monto a depositar debe ser mayor a cero.");
  }
}
```



```
    this.saldo += monto;
}
```

5. Implementación para realizar retiros

- **Código de la prueba (Fase Roja):**

```
JavaScript
const cuenta = new CuentaBancaria("12345", "Carlos Gómez", 700.0);
cuenta.retirar(300.0);
assert.strictEqual(cuenta.obtenerSaldo(), 400.0);
// Error: cuenta.retirar is not a function
```

- **Código de producción (Fase Verde):**

```
JavaScript
// Se agrega dentro de class CuentaBancaria
retirar(monto) {
  if (monto <= 0) {
    throw new Error("El monto a retirar debe ser mayor a cero.");
  }
  if (monto > this.saldo) {
    throw new Error("Fondos insuficientes.");
  }
  this.saldo -= monto;
}
```

6. Implementación para transferir fondos entre cuentas

Para orquestar operaciones entre múltiples cuentas de forma limpia, introducimos la entidad **Banco**.

- **Código de la prueba (Fase Roja):**

```
JavaScript
const { CuentaBancaria, Banco } = require('./cuentaBancaria');

try {
  const banco = new Banco();
  const origen = new CuentaBancaria("111", "Origen", 1000);
  const destino = new CuentaBancaria("222", "Destino", 200);

  banco.registrarCuenta(origen);
```

```

    banco.registrarCuenta(destino);
    banco.transferir("111", "222", 300);

    assert.strictEqual(origen.obtenerSaldo(), 700);
    assert.strictEqual(destino.obtenerSaldo(), 500);
  } catch(error) {
    // Error: Banco is not a constructor
  }
}

```

- **Código de producción actualizado (Fase Verde - unificado en cuentaBancaria.js):**

JavaScript

```

class Banco {
  constructor() {
    this.cuentas = new Map();
  }

  registrarCuenta(cuenta) {
    this.cuentas.set(cuenta.numeroCuenta, cuenta);
  }

  transferir(numeroOrigen, numeroDestino, monto) {
    const cuentaOrigen = this.cuentas.get(numeroOrigen);
    const cuentaDestino = this.cuentas.get(numeroDestino);

    if (!cuentaOrigen) throw new Error("Cuenta origen no encontrada.");
    if (!cuentaDestino) throw new Error("Cuenta destino no encontrada.");

    cuentaOrigen.retirar(monto);
    cuentaDestino.depositar(monto);
  }
}

// Corregido: Exportamos ambas clases en el mismo módulo
module.exports = { CuentaBancaria, Banco };

```

Etapas 3: Pruebas adicionales y mejoras

7 y 8. Pruebas adicionales y verificación (Casos Excepcionales)

Escribimos pruebas dedicadas exclusivamente a verificar que el sistema maneje correctamente los flujos alternativos y lance los errores esperados.

JavaScript

```
// Caso Excepcional A: Intentar retirar más dinero del disponible
const cuentaTest = new CuentaBancaria("999", "Tester", 100);
assert.throws(() => {
  cuentaTest.retirar(150);
}, /Fondos insuficientes/);
```

```
// Caso Excepcional B: Transferir a una cuenta inexistente
const miBanco = new Banco();
const cuentaOrigen = new CuentaBancaria("111", "Juan", 500);
miBanco.registrarCuenta(cuentaOrigen);
```

```
assert.throws(() => {
  miBanco.transferir("111", "999-INEXISTENTE", 100);
}, /Cuenta destino no encontrada/);
```

```
console.log("✓ Pruebas de casos excepcionales superadas.");
```

9 y 10. Refactorización (Refactoring) y re-ejecución de pruebas

- **Problema detectado:** Los métodos **depositar** y **retirar** repiten manualmente la condición **monto <= 0**.
- **Acción de Refactor:** Extraemos esa validación repetida a un método privado (**#validarMontoPositivo**) para limpiar el diseño sin romper el comportamiento público.

Código Refactorizado Final de **cuentaBancaria.js**:

JavaScript

```
class CuentaBancaria {
  constructor(numeroCuenta, titular, saldoInicial = 0) {
    this.numeroCuenta = numeroCuenta;
    this.titular = titular;
    this.saldo = saldoInicial >= 0 ? saldoInicial : 0;
  }

  obtenerSaldo() {
    return this.saldo;
  }

  // Método privado refactorizado para evitar código duplicado
  #validarMontoPositivo(monto) {
    if (monto <= 0) {
```

```

        throw new Error("El monto debe ser mayor a cero.");
    }
}

depositar(monto) {
    this.#validarMontoPositivo(monto);
    this.saldo += monto;
}

retirar(monto) {
    this.#validarMontoPositivo(monto);
    if (monto > this.saldo) {
        throw new Error("Fondos insuficientes.");
    }
    this.saldo -= monto;
}
}

class Banco {
    constructor() {
        this.cuentas = new Map();
    }

    registrarCuenta(cuenta) {
        this.cuentas.set(cuenta.numeroCuenta, cuenta);
    }

    transferir(numeroOrigen, numeroDestino, monto) {
        const cuentaOrigen = this.cuentas.get(numeroOrigen);
        const cuentaDestino = this.cuentas.get(numeroDestino);

        if (!cuentaOrigen) throw new Error("Cuenta origen no encontrada.");
        if (!cuentaDestino) throw new Error("Cuenta destino no encontrada.");

        cuentaOrigen.retirar(monto);
        cuentaDestino.depositar(monto);
    }
}

module.exports = { CuentaBancaria, Banco };

```

Validación: Al volver a ejecutar todas las pruebas escritas hasta el momento, el sistema sigue funcionando en **Verde**, garantizando que el refactor no rompió nada.

Etapas 4: Cobertura completa de pruebas

11 y 12. Cobertura total y análisis de Casos Límite

Para asegurar una robustez del 100%, definimos y probamos los límites del sistema (valores frontera):

1. **Monto cero exacto (0):** El sistema debe impedir depósitos o retiros de \$0 de forma explícita.
 2. **Saldo exacto:** Si una cuenta tiene \$200 y retira exactamente \$200, debe quedar con saldo \$0 sin disparar un error de "fondos insuficientes".
- **Código de pruebas para Casos Límite:**

JavaScript

```
// Límite 1: Bloquear depósitos o retiros con valor 0
const cuentaLimite = new CuentaBancaria("444", "Ana", 200);
assert.throws(() => {
  cuentaLimite.depositar(0);
}, /El monto debe ser mayor a cero/);

assert.throws(() => {
  cuentaLimite.retirar(0);
}, /El monto debe ser mayor a cero/);

// Límite 2: Permitir extracción del total exacto y quedar en cero
cuentaLimite.retirar(200);
assert.strictEqual(cuentaLimite.obtenerSaldo(), 0);
```

13. Ejecución final de la Suite de Pruebas (Fase Verde Completa)

Para validar de forma automatizada y sin depender de dependencias externas (corrigiendo la confusión de usar comandos de Jest con asserts nativos de Node), construimos el script de ejecución final.

Para probar la aplicación completa, creamos el archivo ejecutable **run-tests.js**:

JavaScript

```
// archivo: run-tests.js
const assert = require('assert');
const { CuentaBancaria, Banco } = require('./cuentaBancaria');
```

```

console.log("=== INICIANDO SUITE DE PRUEBAS TDD COMPLETA ===");

try {
  // 1. Caso de éxito básico
  const cuenta = new CuentaBancaria("12345", "Carlos Gómez", 500.0);
  cuenta.depositar(200.0);
  cuenta.retirar(300.0);
  assert.strictEqual(cuenta.obtenerSaldo(), 400.0);

  // 2. Operaciones de Banco y transferencias
  const banco = new Banco();
  const origen = new CuentaBancaria("111", "Origen", 1000);
  const destino = new CuentaBancaria("222", "Destino", 200);
  banco.registrarCuenta(origen);
  banco.registrarCuenta(destino);
  banco.transferir("111", "222", 300);
  assert.strictEqual(origen.obtenerSaldo(), 700);
  assert.strictEqual(destino.obtenerSaldo(), 500);

  // 3. Casos Excepcionales
  assert.throws(() => { origen.retirar(9999); }, /Fondos insuficientes/);
  assert.throws(() => { banco.transferir("111", "999", 50); }, /Cuenta destino no encontrada/);

  // 4. Casos Límite
  assert.throws(() => { origen.depositar(-10); }, /El monto debe ser mayor a cero/);
  assert.throws(() => { origen.depositar(0); }, /El monto debe ser mayor a cero/);

  console.log("\n► Probando Suite Completa... ✓ OK");
  console.log("\n=====");
  console.log("🎉 ¡TODAS LAS PRUEBAS PASARON EXITOSAMENTE!");
  console.log("=====");
} catch (error) {
  console.error("❌ Ocurrió un error en la suite de pruebas:");
  console.error(error.stack);
}

```

Para correr las pruebas y verificar el resultado en consola, simplemente ejecutamos en la terminal:

```

Bash
node run-tests.js

```

Resultado en Consola:

Plaintext

=== INICIANDO SUITE DE PRUEBAS TDD COMPLETA ===

► Probando Suite Completa... ✓ OK

=====

🎉 ¡TODAS LAS PRUEBAS PASARON EXITOSAMENTE!

=====